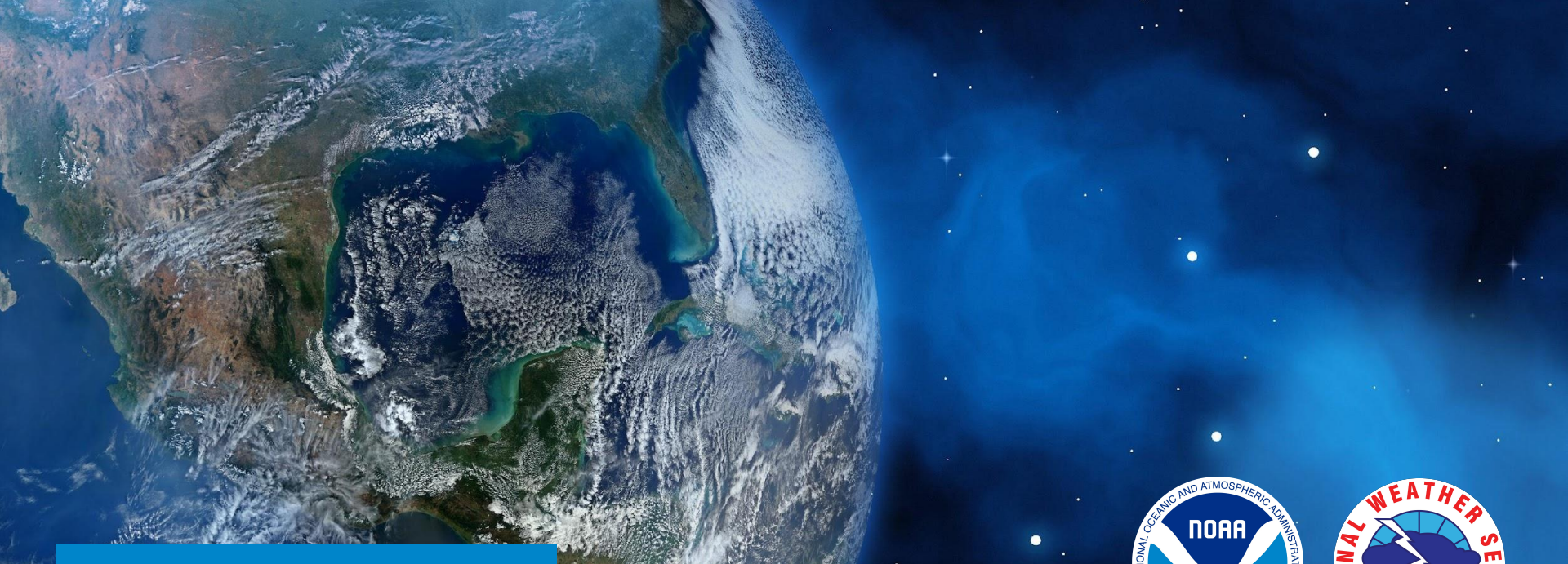


Generative AI Software Development Initiative

Agentic AI-assisted systems for software engineering

Terrence McGuinness / EMC EIB Affiliate



U.S. Department of Commerce
National Oceanic and Atmospheric Administration
National Weather Service

Goals of Generative AI Software Development



- Deliver a platform for easy entry point for developers (VS Code on demand)
- Create tools for automated software development (GitHub Issues)
- Establish automated process for EE2 code compliance recommendations
- Create robust on-the-fly runtime error analysis in CI/CD pipelines
- Stand Up a professional grade Expert System (Ask mode in LLM)

Leveraging State of the Art Agentic AI for Software Development

Provides AI intelligent support for software projects by integrating semantic search, automated compliance reporting, and contextual code assistance directly into software development environments and workflows.

Creating an infrastructure for leveraging generative AI capabilities for agentic code development:

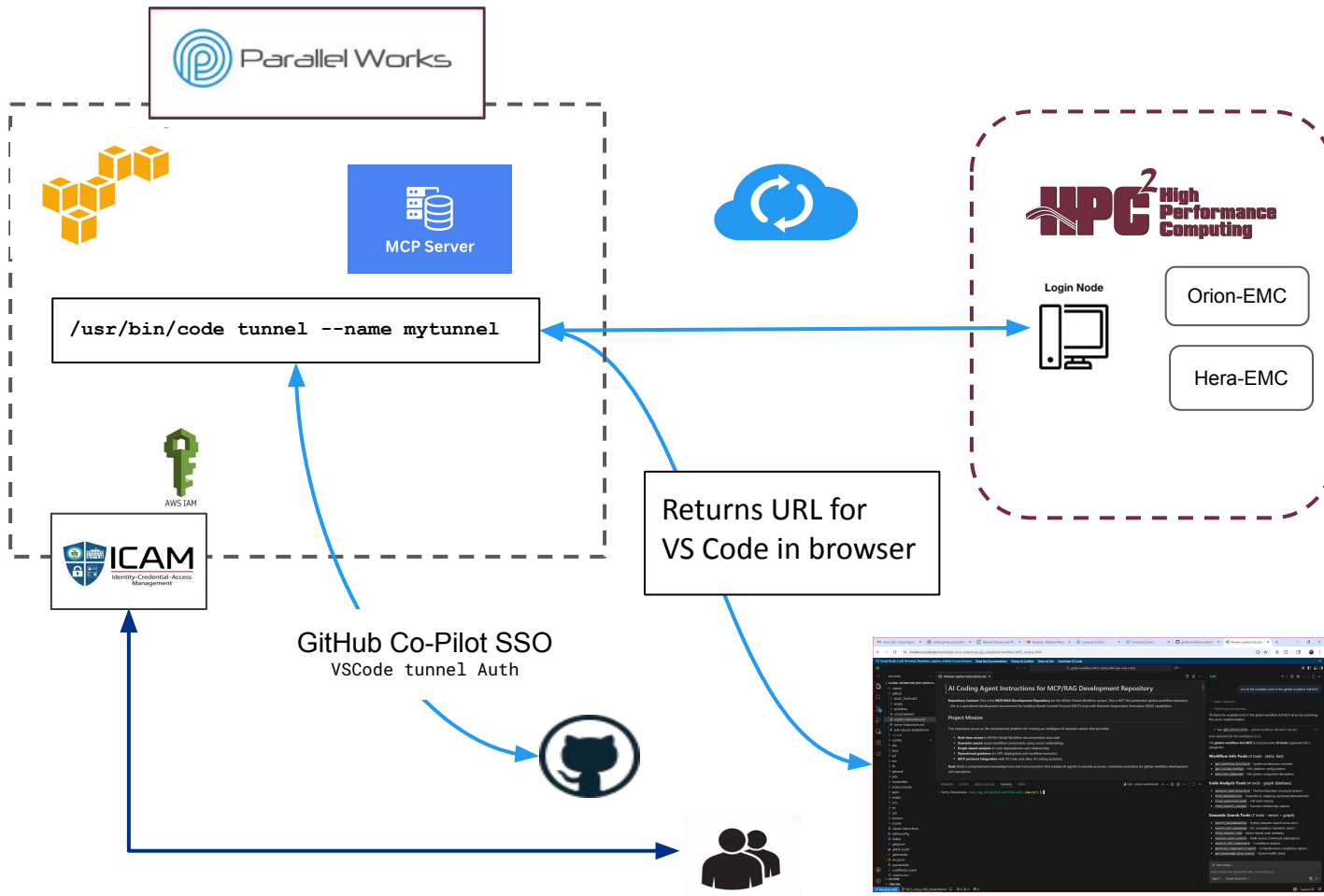
Large Language Models (LLM) are integrated with customized NOAA based **Model Context Protocol (MCP)** tools that integrate seamlessly into coding environments and workflows with **Retrieval-Augmented Generation (RAG)** systems that directly encapsulate specific compliance documents.

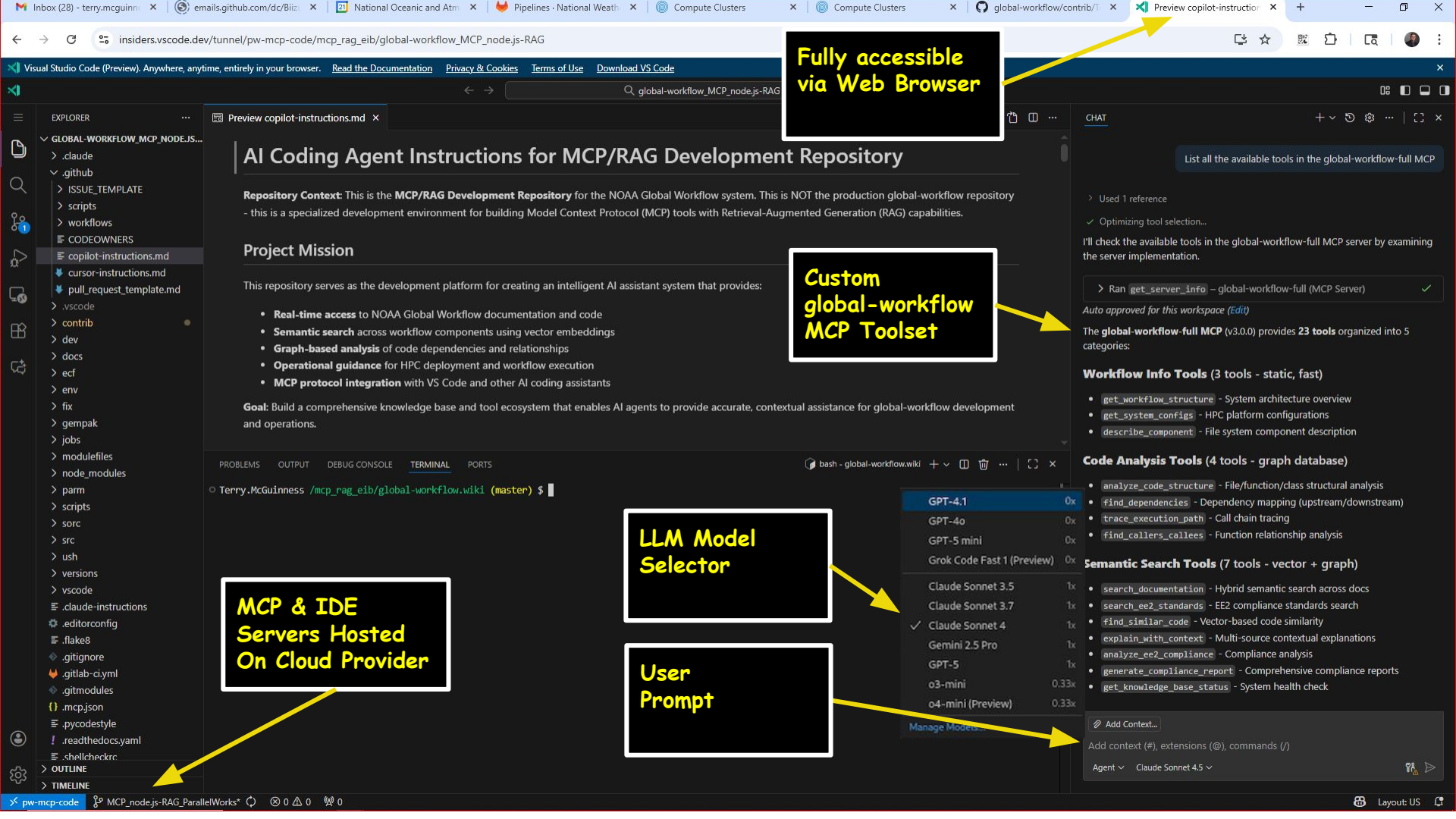
Advantages:

- Provides developers with immediate access to the latest advancements in AI assisted coding development paradigms.
- Empowers advanced subject matter experts by removing the need to perform menial tasks → increasing productivity.
- Creates centralized, searchable repository of critical operational knowledge, reducing reliance on individual expertise.
- Proactively identifies compliance issues in real time, avoiding deployment delays.

*Agentic AI is a class of artificial intelligence that focuses on autonomous systems that can make decisions and perform tasks with or without human intervention. [Wikipedia]

Department of Commerce | National Oceanic and Atmospheric Administration | NOAA.gov





Fully accessible via Web Browser

Custom global-workflow MCP Toolset

LLM Model Selector

User Prompt

MCP & IDE Servers Hosted On Cloud Provider

AI Coding Agent Instructions for MCP/RAG Development Repository

Repository Context: This is the **MCP/RAG Development Repository** for the NOAA Global Workflow system. This is NOT the production global-workflow repository - this is a specialized development environment for building Model Context Protocol (MCP) tools with Retrieval-Augmented Generation (RAG) capabilities.

Project Mission

This repository serves as the development platform for creating an intelligent AI assistant system that provides:

- **Real-time access** to NOAA Global Workflow documentation and code
- **Semantic search** across workflow components using vector embeddings
- **Graph-based analysis** of code dependencies and relationships
- **Operational guidance** for HPC deployment and workflow execution
- **MCP protocol integration** with VS Code and other AI coding assistants

Goal: Build a comprehensive knowledge base and tool ecosystem that enables AI agents to provide accurate, contextual assistance for global-workflow development and operations.

CHAT

List all the available tools in the global-workflow-full MCP

> Used 1 reference

✓ Optimizing tool selection...

I'll check the available tools in the global-workflow-full MCP server by examining the server implementation.

> Ran `get_server_info` - global-workflow-full (MCP Server) ✓

Auto approved for this workspace (Edit)

The **global-workflow-full MCP** (v3.0.0) provides **23 tools** organized into 5 categories:

Workflow Info Tools (3 tools - static, fast)

- `get_workflow_structure` - System architecture overview
- `get_system_configs` - HPC platform configurations
- `describe_component` - File system component description

Code Analysis Tools (4 tools - graph database)

- `analyze_code_structure` - File/function/class structural analysis
- `find_dependencies` - Dependency mapping (upstream/downstream)
- `trace_execution_path` - Call chain tracing
- `find_callers_callees` - Function relationship analysis

Semantic Search Tools (7 tools - vector + graph)

- `search_documentation` - Hybrid semantic search across docs
- `search_ee2_standards` - EE2 compliance standards search
- `find_similar_code` - Vector-based code similarity
- `explain_with_context` - Multi-source contextual explanations
- `analyze_ee2_compliance` - Compliance analysis
- `generate_compliance_report` - Comprehensive compliance reports
- `get_knowledge_base_status` - System health check

+ Add Context...

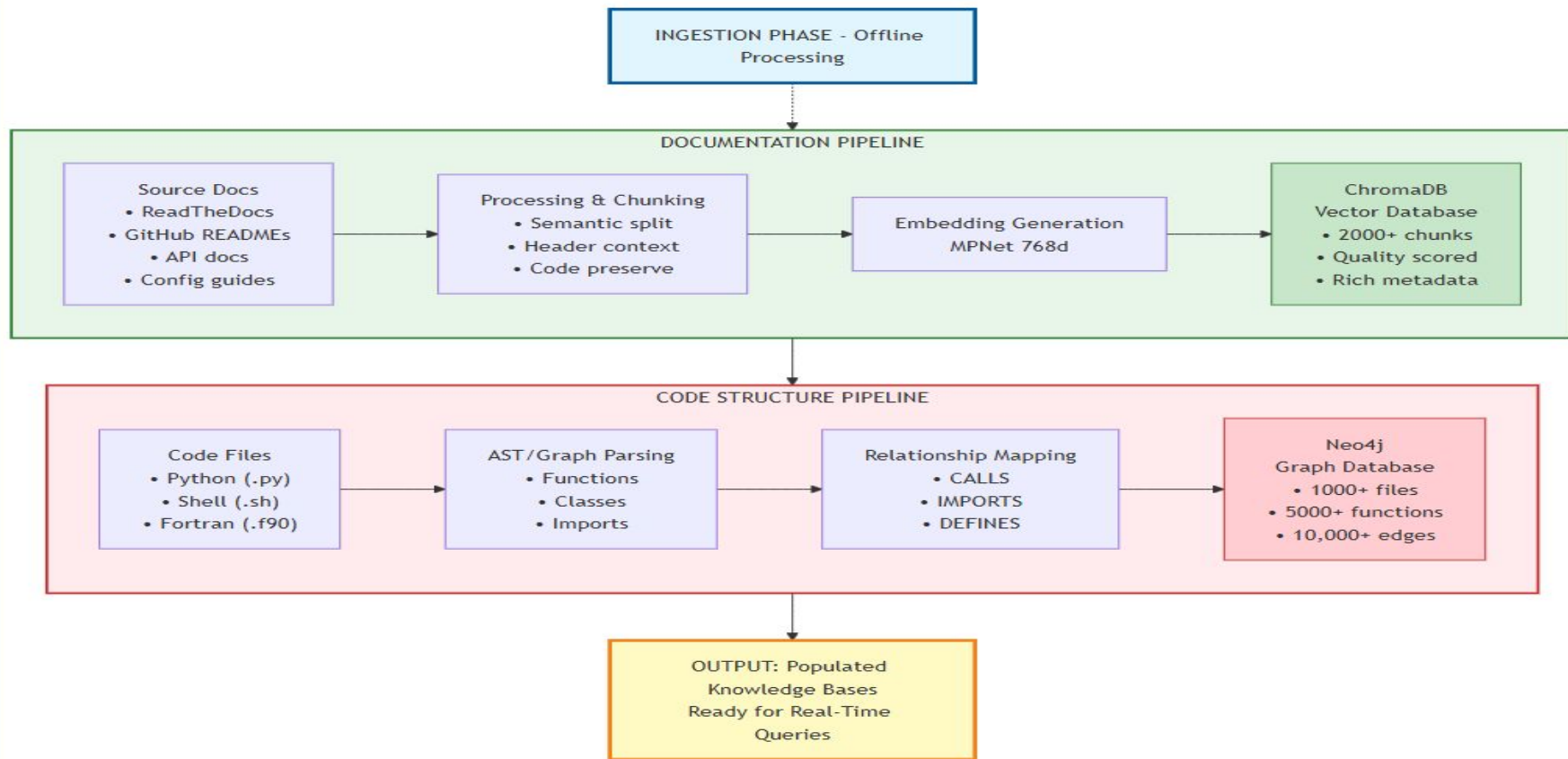
Add context (#), extensions (@), commands (/)

Agent Claude Sonnet 4.5

- GPT-4.1 0x
- GPT-4o 0x
- GPT-5 mini 0x
- Grok Code Fast1 (Preview) 0x
- Claude Sonnet 3.5 1x
- Claude Sonnet 3.7 1x
- ✓ Claude Sonnet 4 1x
- Gemini 2.5 Pro 1x
- GPT-5 1x
- o3-mini 0.33x
- o4-mini (Preview) 0.33x

Manage Models

Retrieval-Augmented Generation (RAG)



Module Context Protocol (MCP) Tool Set [1 of 3]

WorkflowInfoTools (3 tools) - Static/File System

1. `get_workflow_structure` - Get workflow system structure overview
2. `get_system_configs` - Get HPC platform configurations
3. `describe_component` - Basic component description (file-based)

OperationalTools (4 tools) - Graph Database Enhanced

4. `explain_workflow_component` - Deep component explanation with graph data
5. `analyze_workflow_dependencies` - Dependency analysis via graph traversal
6. `get_operational_guidance` - Operational best practices
7. `trace_execution_path` - Execution flow tracing

Module Context Protocol (MCP) Tool Set [2 of 3]

OperationalTools (4 tools) - Graph Database Enhanced

4. `explain_workflow_component` - Deep component explanation with graph data
5. `analyze_workflow_dependencies` - Dependency analysis via graph traversal
6. `get_operational_guidance` - Operational best practices
7. `trace_execution_path` - Execution flow tracing

SemanticSearchTools (4 tools) - Vector Database

8. `search_documentation` - Semantic search across documentation
9. `search_ee2_standards` - Search EE2 compliance standards
10. `explain_with_context` - Context-aware explanations
11. `get_knowledge_base_status` - KB health and statistics

Module Context Protocol (MCP) Tool Set [3 of 3]

CodeAnalysisTools (5 tools) - Graph + Vector

- 12. `analyze_code_structure` - Code structure analysis
- 13. `find_dependencies` - Find code dependencies
- 14. `find_similar_code` - Similarity search
- 15. `find_callers_callees` - Call graph analysis
- 16. `analyze_ee2_compliance` - EE2 compliance checking

GitHubTools (5 tools) - GitHub API

- 17. `analyze_repository_structure` - Repository structure analysis
- 18. `search_issues` - GitHub issues search
- 19. `get_pull_requests` - PR information
- 20. `generate_compliance_report` - Generate compliance reports
- 21. `list_job_scripts` - List workflow job scripts

Generated EE2 Compliance Reports and Implementation Plan

EE2 Compliance Assessment Report

NOAA-EMC/rrfs-workflow Repository

Assessment Date: January 10, 2025

Repository: <https://github.com/NOAA-EMC/rrfs-workflow>

Assessor: GitHub Copilot with MCP/RAG-Enhanced Analysis

Standards Reference: NOAA Environmental Equivalence (EE2) Framework

Executive Summary

The RRFS (Rapid Refresh Forecast System) workflow repository demonstrates **moderate compliance** with EE2 standards, with several areas requiring attention to achieve full operational readiness on WCOSS systems.

Overall Compliance Score: 75/100 ⚠️

Category	Status	Score	Priority
Environment Variables	⚠️ Partial	70%	HIGH
Error Handling	✅ Good	85%	MEDIUM
Workflow Structure	✅ Excellent	95%	LOW
File Naming	✅ Good	90%	LOW
Production Utilities	⚠️ Partial	65%	HIGH
Code Standards	✅ Good	80%	MEDIUM
Directory Structure	✅ Excellent	95%	LOW

Detailed Analysis

1. Environment Variables (70% - Needs Improvement)

✅ Strengths:

- Uses RRFS-specific variables: `WGF`, `ENSMEM_IDX`, `DO_ENSEMBLE`
- Proper cycle date handling with `CDATE`, `PDY`, `cyc`

EE2 Compliance Implementation Plan for RRFS Workflow

Overview

Following the comprehensive EE2 compliance assessment of the NOAA-EMC/rrfs-workflow repository, this issue outlines the implementation plan to bring RRFS workflow components to full EE2 compliance for integration with the global-workflow system.

📊 Assessment Report

Full EE2 Compliance Report: <https://gist.github.com/TerrenceMcGuinness-NOAA/f9c11c5ccb52f6a0cf0b3d12c82da2f4>

Current Compliance Score: 75/100 ⚠️

Target Compliance Score: 95/100 ✅

🔄 Implementation Phases

Phase 1: Critical Fixes (Priority 1) - 2-3 weeks

1.1 Standardize Environment Variables

☐ Replace RRFS-specific variables with EE2 standards

- Convert `HOMErrfs` → `HOMEmodel`
- Convert `USHrrfs` → `USHmodel`
- Convert `EXECrrfs` → `EXECmodel`
- Convert `PARMrrfs` → `PARMmodel`
- Convert `FIXrrfs` → `FIXmodel`

☐ Add missing critical environment variables

```
export DATAROOT=${DATAROOT:-${STMP}/RUNDIRS}
export DATA=${DATA:-${DATAROOT}/${job}}
export COMIN=${COMIN:-${compath.py -o ${NET}/${model_ver}}}
export COMOUT=${COMOUT:-${compath.py -o ${NET}/${model_ver}}}
```

☐ Update all job scripts in `jobs/` directory

- `JRRFS_FORECAST`
- `JRRFS_ANALYSIS_GSI`
- `JRRFS_POST`